CS-GY 6763/CS-UY 3943: Lecture 1
Course introduction, concentration of random variable, applications

Prof. Ainesh Bakshi

Overarching Goals

Goal: Learning how to learn and think critically

Overarching Goals

Goal: Learning how to learn and think critically

In the age of AI, access to information is cheap. Understanding remains hard.

Overarching Goals

Goal: Learning how to learn and think critically

In the age of AI, access to information is cheap. Understanding remains hard.

Material: Understanding the mathematics behind large Machine Learning systems and Data Science

Algorithmic Machine Learning and Data Science

Statistics, machine learning, and data science study how to use data to make better decisions or discoveries.

Algorithmic Machine Learning and Data Science

Statistics, machine learning, and data science study how to use data to make better decisions or discoveries.

In this class, we study how to do so **as quickly as possible, or with limited computational resources.**

Applications by the Numbers

Modern computational systems operate at massive scale:

- **ChatGPT** processes 1 billion queries per day, at at a cost of $700,000+ per day for OpenAI.
- **Google** receives $\approx$ 20,000 Maps queries every second.
- **NASA** collects 6.4 TB of satallite images every day.
- **Rubin Observatory in Chile** will collect 20 TB of images every night.
- **MIT/Harvard Broad Institute** sequences 24 TB of genetic data every day.

Role of Algorithms

Growing demands of data science and machine learning have ushered in a new "golden age" for algorithms research.

- Slowing raw performance increases in CPUs + GPUs.
- Parallelization limited by financial and environmental costs. Currently, data centers account for 5% of US electricity use. Expected to double in next 3 years.

Role of Algorithms

Growing demands of data science and machine learning have ushered in a new "golden age" for algorithms research.

- Slowing raw performance increases in CPUs + GPUs.
- Parallelization limited by financial and environmental costs. Currently, data centers account for 5% of US electricity use. Expected to double in next 3 years.

Typical data applications require combining a diverse set of algorithmic tools. Most are not heavily covered in your traditional algorithms curriculum.

Class Topics

(1) Randomized methods.

(2) Optimization.

(3) Spectral methods (linear algebra) and Fourier methods.

Class Topics

(1) Randomized methods.

(2) Optimization.

(3) Spectral methods (linear algebra) and Fourier methods.

Focus is on teaching tools to design algorithms, not just the algorithms themselves.

Randomized Methods

Section 1: Randomized Algorithms.

It is hard to find an algorithms paper in 2026 that does not use randomness in some way, but this wasn't always the case!

- Probability tools and concentration of random variables (Markovs, Chebyshev, Chernoff/Bernstein inequalities).

Randomized Methods

Section 1: Randomized Algorithms.

It is hard to find an algorithms paper in 2026 that does not use randomness in some way, but this wasn't always the case!

- Probability tools and concentration of random variables (Markovs, Chebyshev, Chernoff/Bernstein inequalities).
- Random hashing for fast data search, load balancing, faster language models, and more. Locality sensitive hashing, MinHash, SimHash, etc.

Randomized Methods

Section 1: Randomized Algorithms.

It is hard to find an algorithms paper in 2026 that does not use randomness in some way, but this wasn't always the case!

- Probability tools and concentration of random variables (Markovs, Chebyshev, Chernoff/Bernstein inequalities).
- Random hashing for fast data search, load balancing, faster language models, and more. Locality sensitive hashing, MinHash, SimHash, etc.
- Sketching and streaming algorithms for compressing and processing data on the fly.

Randomized Methods

Section 1: Randomized Algorithms.

It is hard to find an algorithms paper in 2026 that does not use randomness in some way, but this wasn't always the case!

- Probability tools and concentration of random variables (Markovs, Chebyshev, Chernoff/Bernstein inequalities).
- Random hashing for fast data search, load balancing, faster language models, and more. Locality sensitive hashing, MinHash, SimHash, etc.
- Sketching and streaming algorithms for compressing and processing data on the fly.
- High-dimensional geometry and the Johnson-Lindenstrauss lemma for compressing high dimensional vectors.

Continuous Optimization

Section 2: Optimization.

Optimization has become the algorithmic workhorse of modern machine learning.

- Gradient descent, stochastic gradient descent, coordinate descent, and how to analyze these methods.

Continuous Optimization

Section 2: Optimization.

Optimization has become the algorithmic workhorse of modern machine learning.

- Gradient descent, stochastic gradient descent, coordinate descent, and how to analyze these methods.
- Acceleration, conditioning, preconditioning, adaptive gradient methods.

Continuous Optimization

Section 2: Optimization.

Optimization has become the algorithmic workhorse of modern machine learning.

- Gradient descent, stochastic gradient descent, coordinate descent, and how to analyze these methods.
- Acceleration, conditioning, preconditioning, adaptive gradient methods.
- Constrained optimization, linear programming. Ellipsoid and interior point methods.

Continuous Optimization

Section 2: Optimization.

Optimization has become the algorithmic workhorse of modern machine learning.

- Gradient descent, stochastic gradient descent, coordinate descent, and how to analyze these methods.
- Acceleration, conditioning, preconditioning, adaptive gradient methods.
- Constrained optimization, linear programming. Ellipsoid and interior point methods.
- Discrete optimization, relaxation, submodularity and greedy methods.

Spectral Methods

Section 3: Spectral methods and linear algebra.

"Complex math operations (machine learning, clustering, trend detection) [are] mostly specified as linear algebra on array data" – Michael Stonebraker, Turing Award Winner

- Efficient algorithms for singular value decomposition and eigendecomposition, including randomized methods.

Spectral Methods

Section 3: Spectral methods and linear algebra.

"Complex math operations (machine learning, clustering, trend detection) [are] mostly specified as linear algebra on array data" – Michael Stonebraker, Turing Award Winner

- Efficient algorithms for singular value decomposition and eigendecomposition, including randomized methods.
- Spectral graph theory: i.e. how to use linear algebara to understanding large graphs through linear algebra (social networks, interaction graphs, etc.).

Spectral Methods

Section 3: Spectral methods and linear algebra.

"Complex math operations (machine learning, clustering, trend detection) [are] mostly specified as linear algebra on array data" – Michael Stonebraker, Turing Award Winner

- Efficient algorithms for singular value decomposition and eigendecomposition, including randomized methods.
- Spectral graph theory: i.e. how to use linear algebara to understanding large graphs through linear algebra (social networks, interaction graphs, etc.).
- Spectral clustering and non-linear dimensionality reduction.

Spectral Methods

Section 3: Spectral methods and linear algebra.

"Complex math operations (machine learning, clustering, trend detection) [are] mostly specified as linear algebra on array data" – Michael Stonebraker, Turing Award Winner

- Efficient algorithms for singular value decomposition and eigendecomposition, including randomized methods.
- Spectral graph theory: i.e. how to use linear algebara to understanding large graphs through linear algebra (social networks, interaction graphs, etc.).
- Spectral clustering and non-linear dimensionality reduction.
- Compressed sensing, sparse recovery, and their applications.

Spectral Methods

Section 3: Spectral methods and linear algebra.

"Complex math operations (machine learning, clustering, trend detection) [are] mostly specified as linear algebra on array data" – Michael Stonebraker, Turing Award Winner

- Efficient algorithms for singular value decomposition and eigendecomposition, including randomized methods.
- Spectral graph theory: i.e. how to use linear algebara to understanding large graphs through linear algebra (social networks, interaction graphs, etc.).
- Spectral clustering and non-linear dimensionality reduction.
- Compressed sensing, sparse recovery, and their applications.
- Fast Fourier Transform inspired methods in linear algebra and dimensionality reduction.

What We Won't Cover

Software tools or frameworks. Spark, Torch, Tensorflow, HPC, AWS, etc. If you are interested, CS-GY 6513 might be a good course.

Machine Learning Models + Techniques. Neural nets, genertive models, reinforcement learning, Bayesian methods, unsupervised learning, etc. I assume you have already had a course in ML and the focus of this class is on computational considerations.

But if your research is in machine learning, I think you will find the theoretical tools we learn are more broadly applicable than in designing faster algorithms.

Our Approach

This is primarily a **theory** course.

- Emphasis on proofs of correctness, bounding asymptotic runtimes, convergence analysis, etc. *Why?*

Our Approach

This is primarily a **theory** course.

- Emphasis on proofs of correctness, bounding asymptotic runtimes, convergence analysis, etc. *Why?*
- Learn how to model complex problems in simple ways.

Our Approach

This is primarily a **theory** course.

- Emphasis on proofs of correctness, bounding asymptotic runtimes, convergence analysis, etc. *Why?*
- Learn how to model complex problems in simple ways.
- Learn general mathematical tools that can be applied in a wide variety of problems (in your research, in industry, etc.)

Our Approach

This is primarily a **theory** course.

- Emphasis on proofs of correctness, bounding asymptotic runtimes, convergence analysis, etc. *Why?*
- Learn how to model complex problems in simple ways.
- Learn general mathematical tools that can be applied in a wide variety of problems (in your research, in industry, etc.)
- The homework requires **creative problem solving** and thinking beyond what was covered in class. You will not be able to solve many problems on your first try!

You will need a good background in **probability** and **linear algebra**. See the syllabus for more details. Ask me is you are still unsure.

Course Structure and Logistics

All of this information is on the course webpage `https://www.aineshbakshi.com/amlds-fall-26/` and in the syllabus posted there! Please take a look.

Class structure:

- Lecture once a week.
- Office hours with TAs once a week.
- Office hours with me by appointment.
- Problem solving recitations once a week.

Tech tools:

- **Website** for up-to-date info, lecture notes, readings.
- **Gradescope** for turning in assignments. Sign up using course code.

Course Structure and Logistics

Class work:

- **4 problem sets** (40% of course grade).
  - These are challenging, and the most effective way to learn the material. I recommend you start early, work with others, ask questions on Ed, etc.
  - You must write-up solutions on your own.
- **Midterm (Oct. 27th)** 25% of course grade.

Course Structure and Logistics

Final project or Final exam (25% of grade):

- Final exam will be similar to midterm and problem sets.
- Final project can be based on a recent algorithms paper, and can be either an experimental or theoretical project. Must work in a group of 2 students.
- Example projects and a reading list will be posted soon.

Course Structure and Logistics

Class participation (10% of grade):

- My goal is to know you all individually by the end of this course.
- Lots of ways to earn the full grade: participation in lecture or office hours. Effort on the project.

Important note:

- This is a mixed undergraduate/graduate course.
- Workload is the same, but undergraduates are graded on a different "curve".

Homework Grading Policy

Grading: For each problem you solve completely, clearly indicate it is a complete solution. After submission, you will be asked to explain one randomly selected problem from among your completely solved problems on a whiteboard (Signup sheet for a 10 minute slot each).

- You can use your solution notes as a reference.
- The grade you receive on the explanation will be applied to all of your completely solved problems.
- For problems you do not solve completely, you may write "I don't know" to receive 25% credit.
- Problems that are incomplete without writing "I don't know" receive 0% credit.
- There is no partial credit for incomplete solutions.

Homework Grading Policy: Example

Example: If a problem set has 5 problems and you completely solve 3 of them, one of those 3 will be randomly selected for you to explain.

If you receive 90% on your explanation, you receive 90% on all 3 completely solved problems.

For the remaining 2 problems, you can write "I don't know" on each to receive 25% credit, or 0% if you leave them incomplete.

Your final score would be:

$$\frac{90\% + 90\% + 90\% + 25\% + 25\%}{5} = 64\%$$

Collaboration and Academic Integrity

Collaboration Policy:

- Collaboration is allowed on homework problems, but solutions must be written independently.
- Writing should not be done in parallel.
- You must list all collaborators separately for each problem.

Use of External Results:

- Unless otherwise stated, referencing non-standard theorems and proofs not given in class or previous problems is not allowed.
- All solutions must be proven from first principles.

Course Assistants

Pratyush Avi

pratyushavi@nyu.edu

Nalini Ramanathan

nar8991@nyu.edu

TA Office Hours: Wednesdays 1:00pm–2:30pm, location TBA

Problem-Solving Session: Fridays 12:00pm–1:30pm, Room 1178, 370 Jay Street

Questions?

This Class

Goal: Demonstrate how even the simplest tools from probability can lead to a powerful algorithmic results.

Lecture applications:

- Estimating set size from samples.
- Finding frequent items with small space.

Problem set applications:

- Group testing for diseases (like bird flu, COVID-19, etc.)

Probability Review

Let X be a random variable taking value in some set $\mathcal{S}$. I.e. for a dice, $\mathcal{S} = \{1, \ldots, 6\}$. For a continuous r.v., we might have $\mathcal{S} = \mathbb{R}$.

- **Expectation**: $\mathbb{E}[X] = \sum_{s \in \mathcal{S}} \Pr[X = s] \cdot s$

  For continuous r.v., $\mathbb{E}[X] = \int_{s \in \mathcal{S}} \Pr(s) \cdot s \, ds$.

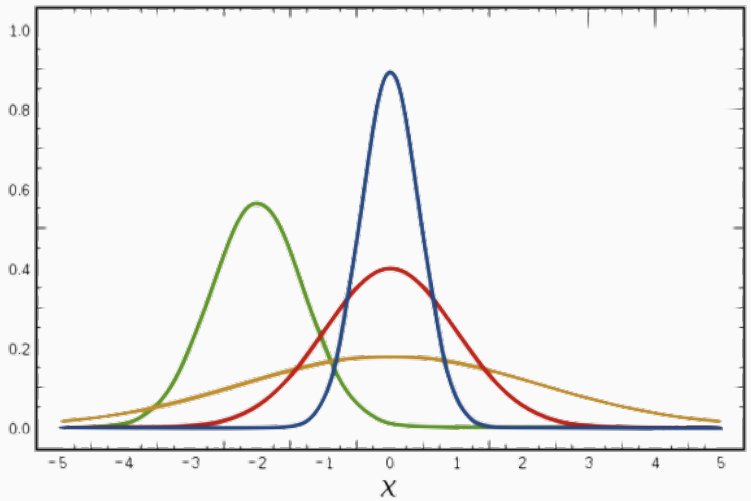
Probability Review

Let X be a random variable taking value in some set $\mathcal{S}$. I.e. for a dice, $\mathcal{S} = \{1, \ldots, 6\}$. For a continuous r.v., we might have $\mathcal{S} = \mathbb{R}$.

- **Expectation**: $\mathbb{E}[X] = \sum_{s \in \mathcal{S}} \Pr[X = s] \cdot s$

  For continuous r.v., $\mathbb{E}[X] = \int_{s \in \mathcal{S}} \Pr(s) \cdot s \, ds$.

- **Variance**: $\mathrm{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$.



For any scalar α, $\mathbb{E}[\alpha X] = \alpha \mathbb{E}[X]$. $\mathrm{Var}[\alpha X] = \alpha^2 \mathrm{Var}[X]$.

Probability Review: Example

Example: Let X be the outcome of rolling a fair die. Compute $\mathbb{E}[X]$ and $\text{Var}[X]$.

Expectation:

Probability Review: Example

Example: Let X be the outcome of rolling a fair die. Compute $\mathbb{E}[X]$ and $\text{Var}[X]$.

Expectation:

$$\mathbb{E}[X] = \sum_{s=1}^{6} \frac{1}{6} \cdot s = \frac{1}{6}(1+2+3+4+5+6) = \frac{21}{6} = 3.5$$

Probability Review: Example

Example: Let X be the outcome of rolling a fair die. Compute $\mathbb{E}[X]$ and $\text{Var}[X]$.

Expectation:

$$\mathbb{E}[X] = \sum_{s=1}^{6} \frac{1}{6} \cdot s = \frac{1}{6}(1+2+3+4+5+6) = \frac{21}{6} = 3.5$$

Probability Review: Example

Example: Let X be the outcome of rolling a fair die. Compute $\mathbb{E}[X]$ and $\text{Var}[X]$.

Expectation:

$$\mathbb{E}[X] = \sum_{s=1}^{6} \frac{1}{6} \cdot s = \frac{1}{6}(1+2+3+4+5+6) = \frac{21}{6} = 3.5$$

Variance: Recall $\text{Var}[X] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$

$$\mathbb{E}[X^2] = \sum_{s=1}^{6} \frac{1}{6} \cdot s^2 = \frac{1}{6}(1+4+9+16+25+36) = \frac{91}{6}$$

Probability Review: Example

Example: Let X be the outcome of rolling a fair die. Compute $\mathbb{E}[X]$ and $\text{Var}[X]$.

Expectation:

$$\mathbb{E}[X] = \sum_{s=1}^{6} \frac{1}{6} \cdot s = \frac{1}{6}(1+2+3+4+5+6) = \frac{21}{6} = 3.5$$

Variance: Recall $\text{Var}[X] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$

$$\mathbb{E}[X^2] = \sum_{s=1}^{6} \frac{1}{6} \cdot s^2 = \frac{1}{6}(1+4+9+16+25+36) = \frac{91}{6}$$

$$\text{Var}[X] = \frac{91}{6} - \left(\frac{7}{2}\right)^2 = \frac{91}{6} - \frac{49}{4} = \frac{182-147}{12} = \frac{35}{12}$$

Probability Review

Let A and B be random events.

- **Joint Probability**: $\Pr(A \cap B)$. Probability that both events happen.
- **Conditional Probability**: $\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)}$. Probability A happens conditioned on the event that B happens.
- **Independence**: A and B are independent events if: $\Pr(A \mid B) = \Pr(A)$.

Alternative definition of independence:

$$\Pr(A \cap B) = \Pr(A) \cdot \Pr(B).$$

Probability Review

Example: What is the probability that for two independent dice rolls taking values uniformly in $\{1, 2, 3, 4, 5, 6\}$, the first roll comes up odd and the second is < 3?

Probability Review

Example: What is the probability that for two independent dice rolls taking values uniformly in $\{1, 2, 3, 4, 5, 6\}$, the first roll comes up odd and the second is < 3?

$$\frac{3}{6} \cdot \frac{2}{6} = \frac{1}{2} \cdot \frac{1}{3} = \frac{1}{6}$$

Let X and Y be random variables. X and Y are independent if, for all events s, t, the random events $[X = s]$ and $[Y = t]$ are independent.

The Most Powerful Theorem in All of Probability?

Linearity of expectation:

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$$

When is this true?

The Most Powerful Theorem in All of Probability?

Linearity of expectation:

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$$

When is this true?

The Most Powerful Theorem in All of Probability?

Linearity of expectation:

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$$

When is this true?

Proof: Let $X \in S$ and $Y \in T$. Then,

$$\mathbb{E}[X + Y] = \sum_{s \in S} \sum_{t \in T} (s + t) \cdot \Pr[X = s, Y = t]$$

The Most Powerful Theorem in All of Probability?

Linearity of expectation:

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$$

When is this true?

Proof: Let $X \in S$ and $Y \in T$. Then,

$$\begin{aligned}\mathbb{E}[X + Y] &= \sum_{s \in S} \sum_{t \in T} (s + t) \cdot \Pr[X = s, Y = t] \\ &= \sum_{s \in S} \sum_{t \in T} s \cdot \Pr[X = s, Y = t] + \sum_{s \in S} \sum_{t \in T} t \cdot \Pr[X = s, Y = t]\end{aligned}$$

The Most Powerful Theorem in All of Probability?

Linearity of expectation:

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$$

When is this true?

Proof: Let $X \in S$ and $Y \in T$. Then,

$$\begin{aligned}
\mathbb{E}[X + Y] &= \sum_{s \in S} \sum_{t \in T} (s + t) \cdot \Pr[X = s, Y = t] \\
&= \sum_{s \in S} \sum_{t \in T} s \cdot \Pr[X = s, Y = t] + \sum_{s \in S} \sum_{t \in T} t \cdot \Pr[X = s, Y = t] \\
&= \sum_{s \in S} s \cdot \sum_{t \in T} \Pr[X = s, Y = t] + \sum_{t \in T} t \cdot \sum_{s \in S} \Pr[X = s, Y = t]
\end{aligned}$$

The Most Powerful Theorem in All of Probability?

Linearity of expectation:

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$$

When is this true?

Proof: Let $X \in S$ and $Y \in T$. Then,

$$\begin{aligned}
\mathbb{E}[X + Y] &= \sum_{s \in S} \sum_{t \in T} (s + t) \cdot \Pr[X = s, Y = t] \\
&= \sum_{s \in S} \sum_{t \in T} s \cdot \Pr[X = s, Y = t] + \sum_{s \in S} \sum_{t \in T} t \cdot \Pr[X = s, Y = t] \\
&= \sum_{s \in S} s \cdot \sum_{t \in T} \Pr[X = s, Y = t] + \sum_{t \in T} t \cdot \sum_{s \in S} \Pr[X = s, Y = t] \\
&= \mathbb{E}[X] + \mathbb{E}[Y]
\end{aligned}$$

Related Equations

Always, sometimes, or never?

For random variables X, Y:

- $\mathbb{E}[XY] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$.

Related Equations

Always, sometimes, or never?

For random variables X, Y:

- $\mathbb{E}[XY] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$.

  Let $\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X]) \cdot (Y - \mathbb{E}[Y])]$.

Related Equations

Always, sometimes, or never?

For random variables X, Y:

- $\mathbb{E}[XY] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$.

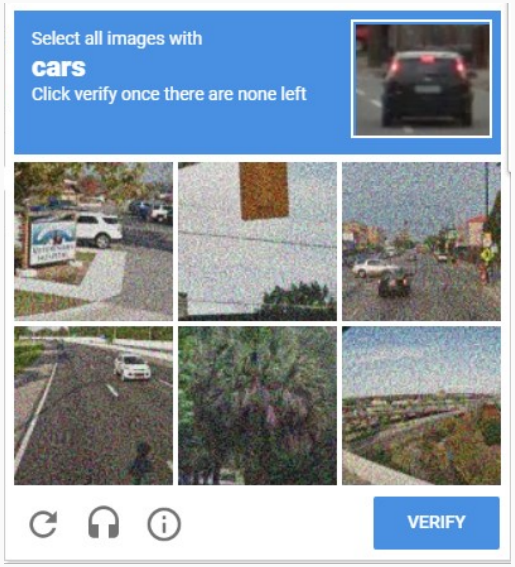
  Let $\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X]) \cdot (Y - \mathbb{E}[Y])]$.
  Then, $\text{Cov}(X, Y) = 0$ is equivalent to $\mathbb{E}[XY] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$.

- $\text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y]$.

  $$\begin{aligned}\text{Var}[X + Y] &= \mathbb{E}\left[(X + Y - \mathbb{E}[X + Y])^2\right] \\ &= \mathbb{E}\left[(X - \mathbb{E}[X])^2 + (Y - \mathbb{E}[Y])^2 + 2 \cdot (X - \mathbb{E}[X]) \cdot (Y - \mathbb{E}[Y])\right] \\ &= \text{Var}[X] + \text{Var}[Y] + 2 \cdot \mathbb{E}\left[(X - \mathbb{E}[X]) \cdot (Y - \mathbb{E}[Y])\right] \\ &= \text{Var}[X] + \text{Var}[Y] + 2 \cdot \text{Cov}(X, Y).\end{aligned}$$

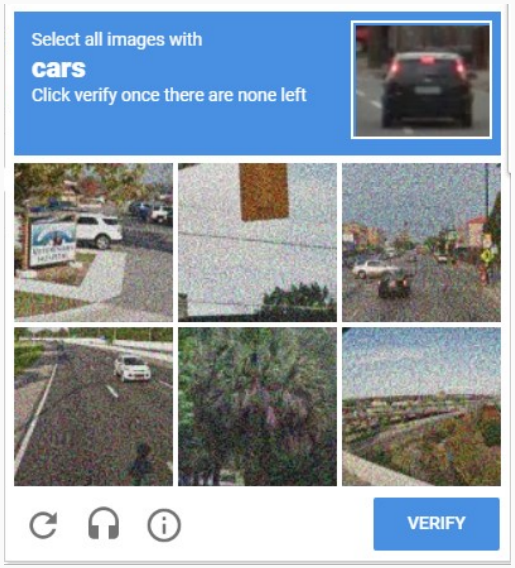
First Application

You run a web company that is considering contracting with a vendor that provides CAPTCHAs for logins.



First Application

You run a web company that is considering contracting with a vendor that provides CAPTCHAs for logins.



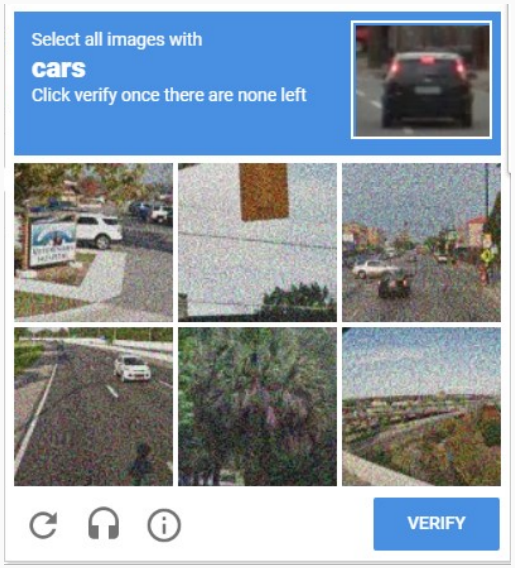
Claim to have a database of $n = \mathbf{1,000,000}$ unique CAPTCHAs.

A random one will be shown on each API call to their service.

They give you access to a test API so you can try it out.

First Application

You run a web company that is considering contracting with a vendor that provides CAPTCHAs for logins.



Claim to have a database of $n = \mathbf{1,000,000}$ unique CAPTCHAs. A random one will be shown on each API call to their service. They give you access to a test API so you can try it out.

Question: Roughly how many queries to the API, m, would you need to independently verify the claim that there are ~ 1 million unique puzzles?

First Application

First attempt: Count how many unique CAPTCHAs you see, until you find $1,000,000$ or close to it. Declare that you are satisfied.

As a function of n, roughly how many API queries m do you need?

First Application

First attempt: Count how many unique CAPTCHAs you see, until you find $1,000,000$ or close to it. Declare that you are satisfied.

As a function of n, roughly how many API queries m do you need?

- At least $\Omega(n)$ queries

First Application

First attempt: Count how many unique CAPTCHAs you see, until you find $1,000,000$ or close to it. Declare that you are satisfied.

As a function of n, roughly how many API queries m do you need?

- At least $\Omega(n)$ queries
- By coupon collector (2 lectures from now) $m = O(n \log(n))$ suffices to see each unique CAPTCHA.

First Application

First attempt: Count how many unique CAPTCHAs you see, until you find $1,000,000$ or close to it. Declare that you are satisfied.

As a function of n, roughly how many API queries m do you need?

- At least $\Omega(n)$ queries
- By coupon collector (2 lectures from now) $m = O(n \log(n))$ suffices to see each unique CAPTCHA.
- Today: $O(\sqrt{n})$ queries suffice! Randomized verification.

A Different Approach

Clever alternative: Count how many duplicate CAPTCHAs you see.

A Different Approach

Clever alternative: Count how many duplicate CAPTCHAs you see.

Key Idea: If the the database has many unique CAPTCHAs, you should see very few duplicates. If there are few unique CAPTCHAs, you should see many duplicates.

If you see the same CAPTCHA on query i and j, that's one duplicate. If you see the same CAPTCHA on queries i, j, and k, that's three duplicates: (i, j), (i, k), (j, k).

Formalizing the Problem

Question: How many duplicates do we expect to see if I have n unique CAPTCHAs?

Formalizing the Problem

Question: How many duplicates do we expect to see if I have n unique CAPTCHAs?

Let $D_{i,j} = 1$ if queries i, j return the same CAPTCHA, and 0 otherwise.

Formalizing the Problem

Question: How many duplicates do we expect to see if I have n unique CAPTCHAs?

Let $D_{i,j} = 1$ if queries i, j return the same CAPTCHA, and 0 otherwise. This is called an indicator random variable.

Formalizing the Problem

Question: How many duplicates do we expect to see if I have n unique CAPTCHAs?

Let $D_{i,j} = 1$ if queries i, j return the same CAPTCHA, and 0 otherwise. This is called an indicator random variable.

$$D_{i,j} = \mathbb{1}[\text{CAPTCHA } i \text{ equals CAPTCHA } j]$$

Number of duplicates D is :

Formalizing the Problem

Question: How many duplicates do we expect to see if I have n unique CAPTCHAs?

Let $D_{i,j} = 1$ if queries i, j return the same CAPTCHA, and 0 otherwise. This is called an indicator random variable.

$$D_{i,j} = \mathbb{1}[\text{CAPTCHA } i \text{ equals CAPTCHA } j]$$

Number of duplicates D is :

$$D = \sum_{\substack{i,j \in \{1,\ldots,m\} \\ i<j}} D_{i,j}.$$

Formalizing the Problem

Question: How many duplicates do we expect to see if I have n unique CAPTCHAs?

Let $D_{i,j} = 1$ if queries i, j return the same CAPTCHA, and 0 otherwise. This is called an indicator random variable.

$$D_{i,j} = \mathbb{1}[\text{CAPTCHA } i \text{ equals CAPTCHA } j]$$

Number of duplicates D is :

$$D = \sum_{\substack{i,j \in \{1,\ldots,m\} \\ i<j}} D_{i,j}.$$

Formalizing the Problem

Question: How many duplicates do we expect to see if I have n unique CAPTCHAs?

Let $D_{i,j} = 1$ if queries i, j return the same CAPTCHA, and 0 otherwise. This is called an indicator random variable.

$$D_{i,j} = \mathbb{1}[\text{CAPTCHA } i \text{ equals CAPTCHA } j]$$

Number of duplicates D is :

$$D = \sum_{\substack{i,j \in \{1,\ldots,m\} \\ i<j}} D_{i,j}.$$

What is $\mathbb{E}[D]$?

Formalizing the Problem

Question: How many duplicates do we expect to see? Formally, what is $\mathbb{E}[D]$?

$$\mathbb{E}[D] = \mathbb{E}\left[\sum_{\substack{i,j \in \{1,\ldots,m\} \\ i<j}} D_{i,j}\right] = \sum_{\substack{i,j \in \{1,\ldots,m\} \\ i<j}} \mathbb{E}[D_{i,j}]$$

n = number of CAPTCHAS in database, m = number of test queries.
$D_{i,j}$ = indicator for event CAPTCHA i and j collide.

Formalizing the Problem

Question: How many duplicates do we expect to see? Formally, what is $\mathbb{E}[D]$?

$$\mathbb{E}[D] = \mathbb{E}\left[\sum_{\substack{i,j\in\{1,\ldots,m\}\\ i<j}} D_{i,j}\right] = \sum_{\substack{i,j\in\{1,\ldots,m\}\\ i<j}} \mathbb{E}[D_{i,j}]$$

$$= \sum_{\substack{i,j\in\{1,\ldots,m\}\\ i<j}} \Pr[\text{CAPTCHA } i = \text{CAPTCHA } j] = \sum_{\substack{i,j\in\{1,\ldots,m\}\\ i<j}} \frac{1}{n}$$

n = number of CAPTCHAS in database, m = number of test queries.
$D_{i,j}$ = indicator for event CAPTCHA i and j collide.

Formalizing the Problem

Question: How many duplicates do we expect to see? Formally, what is $\mathbb{E}[D]$?

$$\mathbb{E}[D] = \mathbb{E}\left[\sum_{\substack{i,j\in\{1,\ldots,m\}\\ i<j}} D_{i,j}\right] = \sum_{\substack{i,j\in\{1,\ldots,m\}\\ i<j}} \mathbb{E}[D_{i,j}]$$

$$= \sum_{\substack{i,j\in\{1,\ldots,m\}\\ i<j}} \Pr[\text{CAPTCHA } i = \text{CAPTCHA } j] = \sum_{\substack{i,j\in\{1,\ldots,m\}\\ i<j}} \frac{1}{n}$$

$$= \binom{m}{2} \cdot \frac{1}{n} = \frac{m(m-1)}{2n}$$

n = number of CAPTCHAS in database, m = number of test queries.
$D_{i,j}$ = indicator for event CAPTCHA i and j collide.

Some Hard Numbers

Suppose you take $m = 1000$ queries and see 10 duplicates. How does this compare to the expectation if the database actually has $n = 1,000,000$ unique CAPTCHAs?

Some Hard Numbers

Suppose you take $m = 1000$ queries and see 10 duplicates. How does this compare to the expectation if the database actually has $n = 1,000,000$ unique CAPTCHAs?

$$\mathbb{E}[D] = \frac{m(m-1)}{2n} = \frac{1000 \cdot 999}{2 \cdot 1{,}000{,}000}$$

Some Hard Numbers

Suppose you take $m = 1000$ queries and see 10 duplicates. How does this compare to the expectation if the database actually has $n = 1,000,000$ unique CAPTCHAs?

$$\mathbb{E}[D] = \frac{m(m-1)}{2n} = \frac{1000 \cdot 999}{2 \cdot 1{,}000{,}000}$$
$$= \frac{999{,}000}{2{,}000{,}000} \approx 0.5$$

Some Hard Numbers

Suppose you take $m = 1000$ queries and see 10 duplicates. How does this compare to the expectation if the database actually has $n = 1,000,000$ unique CAPTCHAs?

$$\mathbb{E}[D] = \frac{m(m-1)}{2n} = \frac{1000 \cdot 999}{2 \cdot 1{,}000{,}000}$$
$$= \frac{999{,}000}{2{,}000{,}000} \approx 0.5$$

Some Hard Numbers

Suppose you take $m = 1000$ queries and see 10 duplicates. How does this compare to the expectation if the database actually has $n = 1,000,000$ unique CAPTCHAs?

$$\mathbb{E}[D] = \frac{m(m-1)}{2n} = \frac{1000 \cdot 999}{2 \cdot 1{,}000{,}000}$$
$$= \frac{999{,}000}{2{,}000{,}000} \approx 0.5$$

Something seems wrong... this random variable D came up much larger than it's expectation.

Can we say something formally?

n = number of CAPTCHAS in database, m = number of test queries.

Concentration Inequality

One of the most important tools in analyzing randomized algorithms. Tell us how likely it is that a random variable X deviates a certain amount from its expectation $\mathbb{E}[X]$.

Concentration Inequality

One of the most important tools in analyzing randomized algorithms. Tell us how likely it is that a random variable X deviates a certain amount from its expectation $\mathbb{E}[X]$. We will learn three fundamental concentration inequalities:

1. **Markov's Inequality**.
    - Applies to non-negative random variables.
2. Chebyshev's Inequality.
    - Applies to random variables with bounded variance.
3. Hoeffding/Bernstein/Chernoff bounds.
    - Apply to sums of independent random variables.

Markov's Inequality

Theorem (Markov's Inequality): For any random variable X which only takes non-negative values, and any positive t,

$$\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}.$$

Equivalently, for any $\alpha > 0$,

Proof:

Markov's Inequality

Theorem (Markov's Inequality): For any random variable X which only takes non-negative values, and any positive t,

$$\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}.$$

Equivalently, for any $\alpha > 0$,

$$\Pr[X \geq \alpha \cdot \mathbb{E}[X]] \leq \frac{1}{\alpha}.$$

Proof:

Markov's Inequality

Theorem (Markov's Inequality): For any random variable X which only takes non-negative values, and any positive t,

$$\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}.$$

Equivalently, for any $\alpha > 0$,

$$\Pr[X \geq \alpha \cdot \mathbb{E}[X]] \leq \frac{1}{\alpha}.$$

Proof:

Markov's Inequality

Theorem (Markov's Inequality): For any random variable X which only takes non-negative values, and any positive t,

$$\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}.$$

Equivalently, for any $\alpha > 0$,

$$\Pr[X \geq \alpha \cdot \mathbb{E}[X]] \leq \frac{1}{\alpha}.$$

Proof:

$$\mathbb{E}[X] = \mathbb{E}[X \cdot \mathbb{1}_{X<t}] + \mathbb{E}[X \cdot \mathbb{1}_{X \geq t}]$$

Therefore, $\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}$. □

Markov's Inequality

Theorem (Markov's Inequality): For any random variable X which only takes non-negative values, and any positive t,

$$\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}.$$

Equivalently, for any $\alpha > 0$,

$$\Pr[X \geq \alpha \cdot \mathbb{E}[X]] \leq \frac{1}{\alpha}.$$

Proof:

$$\begin{aligned}\mathbb{E}[X] &= \mathbb{E}[X \cdot \mathbb{1}_{X<t}] + \mathbb{E}[X \cdot \mathbb{1}_{X\geq t}] \\ &\geq \mathbb{E}[X \cdot \mathbb{1}_{X\geq t}] \quad (\text{since } X \geq 0)\end{aligned}$$

Therefore, $\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}$. □

Markov's Inequality

Theorem (Markov's Inequality): For any random variable X which only takes non-negative values, and any positive t,

$$\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}.$$

Equivalently, for any $\alpha > 0$,

$$\Pr[X \geq \alpha \cdot \mathbb{E}[X]] \leq \frac{1}{\alpha}.$$

Proof:

$$\begin{aligned}
\mathbb{E}[X] &= \mathbb{E}[X \cdot \mathbb{1}_{X<t}] + \mathbb{E}[X \cdot \mathbb{1}_{X\geq t}] \\
&\geq \mathbb{E}[X \cdot \mathbb{1}_{X\geq t}] \quad (\text{since } X \geq 0) \\
&\geq \mathbb{E}[t \cdot \mathbb{1}_{X\geq t}] \quad (\text{since } X \geq t \text{ when indicator is 1})
\end{aligned}$$

Therefore, $\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}$. □

Markov's Inequality

Theorem (Markov's Inequality): For any random variable X which only takes non-negative values, and any positive t,

$$\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}.$$

Equivalently, for any $\alpha > 0$,

$$\Pr[X \geq \alpha \cdot \mathbb{E}[X]] \leq \frac{1}{\alpha}.$$

Proof:

$$\begin{aligned}
\mathbb{E}[X] &= \mathbb{E}[X \cdot \mathbb{1}_{X<t}] + \mathbb{E}[X \cdot \mathbb{1}_{X\geq t}] \\
&\geq \mathbb{E}[X \cdot \mathbb{1}_{X\geq t}] \quad (\text{since } X \geq 0) \\
&\geq \mathbb{E}[t \cdot \mathbb{1}_{X\geq t}] \quad (\text{since } X \geq t \text{ when indicator is 1}) \\
&= t \cdot \Pr[X \geq t]
\end{aligned}$$

Therefore, $\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}$. □

Application to CAPTCHA Problem

Suppose you take $m = 1000$ queries and see 10 duplicates. How does this compare to the expectation if the database actually has $n = 1,000,000$ unique CAPTCHAs?

$$\mathbb{E}[D] = \frac{m(m-1)}{2n} = .4995.$$

Application to CAPTCHA Problem

Suppose you take $m = 1000$ queries and see 10 duplicates. How does this compare to the expectation if the database actually has $n = 1,000,000$ unique CAPTCHAs?

$$\mathbb{E}[D] = \frac{m(m-1)}{2n} = .4995.$$

Application to CAPTCHA Problem

Suppose you take $m = 1000$ queries and see 10 duplicates. How does this compare to the expectation if the database actually has $n = 1,000,000$ unique CAPTCHAs?

$$\mathbb{E}[D] = \frac{m(m-1)}{2n} = .4995.$$

By Markov's:

$$\Pr[D \geq 10] \leq \frac{\mathbb{E}[D]}{10} < .05 \text{ if } n \text{ actually equals 1 million.}$$

We can be pretty sure we're being scammed...

n = number of CAPTCHAS in database, m = number of test queries.

General Bound

Alternative view: If $\mathbb{E}[D] = \frac{m(m-1)}{2n}$, then a natural estimator for n is:

$$\tilde{n} = \frac{m(m-1)}{2D}.$$

General Bound

Alternative view: If $\mathbb{E}[D] = \frac{m(m-1)}{2n}$, then a natural estimator for n is:

$$\tilde{n} = \frac{m(m-1)}{2D}.$$

General Bound

Alternative view: If $\mathbb{E}[D] = \frac{m(m-1)}{2n}$, then a natural estimator for n is:

$$\tilde{n} = \frac{m(m-1)}{2D}.$$

With a little more work it is possible to show the following:

Claim: If $m = O\left(\frac{\sqrt{n}}{\epsilon}\right)$, then with probability $9/10$, $(1-\epsilon)n \leq \tilde{n} \leq (1+\epsilon)n$. This is a two-sided multiplicative error guarantee. **You will prove this on homework.**

General Bound

Alternative view: If $\mathbb{E}[D] = \frac{m(m-1)}{2n}$, then a natural estimator for n is:

$$\tilde{n} = \frac{m(m-1)}{2D}.$$

With a little more work it is possible to show the following:

Claim: If $m = O\left(\frac{\sqrt{n}}{\epsilon}\right)$, then with probability $9/10$, $(1-\epsilon)n \leq \tilde{n} \leq (1+\epsilon)n$. This is a two-sided multiplicative error guarantee. **You will prove this on homework.**

This is a lot better than our original method that required $O(n)$ queries!

Mark and Recapture

Fun facts:

- Known as the "mark-and-recapture" method in ecology.
- Can also be used by webcrawlers to estimate the size of the internet, a social network, etc.

This is also closely related to the birthday paradox.

First Set of Tools

Linearity of Expectation + Markov's Inequality

Primitive but powerful toolkit, which can be applied to a wide variety of applications!

The Frequent Items Problem

k-Frequent Items (Heavy-Hitters) Problem: Consider a stream of n items $x_1, \ldots, x_n$ with duplicates. These items take u possible values. Return any item at appears at least $\frac{n}{k}$ times.

The Frequent Items Problem

k-Frequent Items (Heavy-Hitters) Problem: Consider a stream of n items $x_1, \ldots, x_n$ with duplicates. These items take u possible values. Return any item at appears at least $\frac{n}{k}$ times.

$x_1,\ x_2,\ x_3, x_4, x_5,\ x_6, \ldots$

The Frequent Items Problem

k-Frequent Items (Heavy-Hitters) Problem: Consider a stream of n items $x_1, \ldots, x_n$ with duplicates. These items take u possible values. Return any item at appears at least $\frac{n}{k}$ times.

$x_1,\ x_2,\ x_3, x_4, x_5,\ x_6, \ldots$

$3, 8, 10, 4, 3, 2 \ldots$

The Frequent Items Problem

k-Frequent Items (Heavy-Hitters) Problem: Consider a stream of n items $x_1, \ldots, x_n$ with duplicates. These items take u possible values. Return any item at appears at least $\frac{n}{k}$ times.

$x_1,\ x_2,\ x_3, x_4, x_5,\ x_6, \ldots$

$3, 8, 10, 4, 3, 2 \ldots$

The Frequent Items Problem

k-Frequent Items (Heavy-Hitters) Problem: Consider a stream of n items $x_1, \ldots, x_n$ with duplicates. These items take u possible values. Return any item at appears at least $\frac{n}{k}$ times.

$x_1,\ x_2,\ x_3, x_4, x_5,\ x_6, \ldots$

$3, 8, 10, 4, 3, 2 \ldots$

- Finding top/viral items (i.e., products on Amazon, videos watched on Youtube, Google searches, etc.)
- Finding very frequent IP addresses sending requests (to detect DoS attacks/network anomalies).
- 'Iceberg queries' for all items in a database with frequency above some threshold.

Want very fast detection, without having to scan through database/logs. I.e., want to maintain a running list of frequent items that appear in a stream of data items.

The Frequent Items Problem

k-**Frequent Items (Heavy-Hitters) Problem**: Consider a stream of n items $x_1, \ldots, x_n$ with duplicates. These items take u possible values. Return any item that appears at least $\frac{n}{k}$ times.

The Frequent Items Problem

k-**Frequent Items (Heavy-Hitters) Problem**: Consider a stream of n items $x_1, \ldots, x_n$ with duplicates. These items take u possible values. Return any item that appears at least $\frac{n}{k}$ times.

Let v_i be the number of times item i appears in the stream (frequency of item i)

The Frequent Items Problem

k-Frequent Items (Heavy-Hitters) Problem: Consider a stream of n items $x_1, \ldots, x_n$ with duplicates. These items take u possible values. Return any item that appears at least $\frac{n}{k}$ times.

Let v_i be the number of times item i appears in the stream (frequency of item i)

v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9
5	12	3	3	4	5	5	10	3

The Frequent Items Problem

k-Frequent Items (Heavy-Hitters) Problem: Consider a stream of n items $x_1, \ldots, x_n$ with duplicates. These items take u possible values. Return any item that appears at least $\frac{n}{k}$ times.

Let v_i be the number of times item i appears in the stream (frequency of item i)

v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9
5	12	3	3	4	5	5	10	3

- Trivial with $O(u)$ space – store the count for each item and return the one that appears $\geq n/k$ times.

Frequent Subset Mining

Example where linear dependence on u is too large: Find common subsets within a collection of sets. Each subset is an "item".

Frequent Subset Mining

Example where linear dependence on u is too large: Find common subsets within a collection of sets. Each subset is an "item".

- For product recommendations, the number of pairs of products might grow quadratically with the number of products. Amazon has 12 million products. (12 million) $\times$ 4 bytes $=$ 48 megabytes .
(12 million)^2 $\times$ 4 bytes $=$ 576 terabytes to maintain counts.

Frequent Subset Mining

Example where linear dependence on u is too large: Find common subsets within a collection of sets. Each subset is an "item".

- For product recommendations, the number of pairs of products might grow quadratically with the number of products. Amazon has 12 million products. (12 million) $\times$ 4 bytes $=$ 48 megabytes . (12 million)^2 $\times$ 4 bytes $=$ 576 terabytes to maintain counts.
- For social media recommendations, we might have a set of followers for each user and want to count frequent subsets of who they follow. Even higher complexity.

Approximate Frequent Elements

Issue: Can prove that no algorithm using $o(u)$ space can output just the items with frequency $\geq n/k$. We will only be able to solve the problem approximately.

Approximate Frequent Elements

Issue: Can prove that no algorithm using $o(u)$ space can output just the items with frequency $\geq n/k$. We will only be able to solve the problem approximately.

(ϵ, k)-**Frequent Items Problem**: Consider a stream of n items $x_1, \ldots, x_n$. Return a set of items F, including all items that appear $\geq \frac{n}{k}$ times and only items that appear $\geq (1-\epsilon) \cdot \frac{n}{k}$ times.

Approximate Frequent Elements

Issue: Can prove that no algorithm using $o(u)$ space can output just the items with frequency $\geq n/k$. We will only be able to solve the problem approximately.

(ϵ, k)-**Frequent Items Problem**: Consider a stream of n items $x_1, \ldots, x_n$. Return a set of items F, including all items that appear $\geq \frac{n}{k}$ times and only items that appear $\geq (1-\epsilon) \cdot \frac{n}{k}$ times.

The deterministic **Misra-Gries algorithm** solves this problem using $O(k/\epsilon)$ space. We will see a randomized algorithm that matches this, and is more flexible in many settings.

Frequent Elements with Count-Min Sketch

Today: Count-Min Sketch – a random hashing based method for the frequent elements problem.

Frequent Elements with Count-Min Sketch

Today: Count-Min Sketch – a random hashing based method for the frequent elements problem.

Due to a 2005 paper by Graham Cormode and Muthu Muthukrishnan.

Solves the slightly different **point query** problem. Given any value v, let $f(v) = \sum_{i=1}^{n} \mathbb{1}[x_i = v]$ be the number of times v appears in the stream.

Frequent Elements with Count-Min Sketch

Today: Count-Min Sketch – a random hashing based method for the frequent elements problem.

Due to a 2005 paper by Graham Cormode and Muthu Muthukrishnan.

Solves the slightly different **point query** problem. Given any value v, let $f(v) = \sum_{i=1}^{n} \mathbb{1}[x_i = v]$ be the number of times v appears in the stream.

Goal: Return estimate $\tilde{f}(v)$ such that $f(v) \leq \tilde{f}(v) \leq f(v) + \epsilon \cdot \frac{n}{k}$ with high probability.

Frequent Elements with Count-Min Sketch

Today: Count-Min Sketch – a random hashing based method for the frequent elements problem.

Due to a 2005 paper by Graham Cormode and Muthu Muthukrishnan.

Solves the slightly different **point query** problem. Given any value v, let $f(v) = \sum_{i=1}^{n} \mathbb{1}[x_i = v]$ be the number of times v appears in the stream.

Goal: Return estimate $\tilde{f}(v)$ such that $f(v) \leq \tilde{f}(v) \leq f(v) + \epsilon \cdot \frac{n}{k}$ with high probability.

Solving Frequent items: Just return all items for which $\tilde{f}(v) \geq \frac{n}{k}$.

Frequent Elements with Count-Min Sketch

Today: Count-Min Sketch – a random hashing based method for the frequent elements problem.

Due to a 2005 paper by Graham Cormode and Muthu Muthukrishnan.

Solves the slightly different **point query** problem. Given any value v, let $f(v) = \sum_{i=1}^{n} \mathbb{1}[x_i = v]$ be the number of times v appears in the stream.

Goal: Return estimate $\tilde{f}(v)$ such that $f(v) \leq \tilde{f}(v) \leq f(v) + \epsilon \cdot \frac{n}{k}$ with high probability.

Solving Frequent items: Just return all items for which $\tilde{f}(v) \geq \frac{n}{k}$.

Assume $f(v) < (1 - \epsilon) \cdot \frac{n}{k}$. Then $\tilde{f}(v) \leq f(v) + \epsilon \cdot \frac{n}{k} < \frac{n}{k}$.

Random Hash Function

Let h be a random function from $\mathcal{U} = \{1, \ldots, u\} \to \{1, \ldots, m\}$. This means that h is constructed by an algorithm using a seed of random numbers, but then the function is fixed. Given input $x \in \mathcal{U}$, it always returns the same output, $h(x)$.

Random Hash Function

Let h be a random function from $\mathcal{U} = \{1, \ldots, u\} \to \{1, \ldots, m\}$. This means that h is constructed by an algorithm using a seed of random numbers, but then the function is fixed. Given input $x \in \mathcal{U}$, it always returns the same output, $h(x)$.

Definition: Uniformly Random Hash Function. A random function $h : \mathcal{U} \to \{1, \ldots, m\}$ is called uniformly random if:

- $\Pr[h(x) = i] = \frac{1}{m}$ for all $x \in \mathcal{U}$, $i \in \{1, \ldots, m\}$.
- $h(x)$ and $h(y)$ are independent r.v.'s for all $x, y \in \mathcal{U}$.

Random Hash Function

Let h be a random function from $\mathcal{U} = \{1, \ldots, u\} \to \{1, \ldots, m\}$. This means that h is constructed by an algorithm using a seed of random numbers, but then the function is fixed. Given input $x \in \mathcal{U}$, it always returns the same output, $h(x)$.

Definition: Uniformly Random Hash Function. A random function $h : \mathcal{U} \to \{1, \ldots, m\}$ is called uniformly random if:

- $\Pr[h(x) = i] = \frac{1}{m}$ for all $x \in \mathcal{U}$, $i \in \{1, \ldots, m\}$.
- $h(x)$ and $h(y)$ are independent r.v.'s for all $x, y \in \mathcal{U}$.
    - Which implies that $\Pr[h(x) = h(y)] = \frac{1}{m}$

$\mathcal{U}$ = universe of possible keys, m = number of values hashed to.

Random Hash Function

Caveat: It is not possible to efficiently implement uniform random hash functions! (Even if we have access to truly random numbers)
But:

Random Hash Function

Caveat: It is not possible to efficiently implement uniform random hash functions! (Even if we have access to truly random numbers)
But:

- In practice "random looking" functions suffice.
- We will discuss weaker, efficiently implementable hash functions (in particular, universal hash functions) next week.
- Our analysis will work with these weaker hash functions.

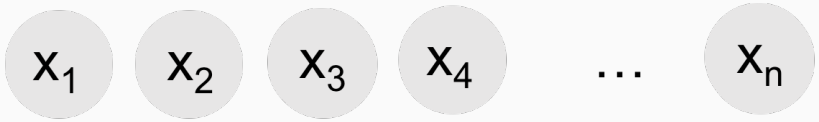
Random Hash Function

Caveat: It is not possible to efficiently implement uniform random hash functions! (Even if we have access to truly random numbers) But:

- In practice "random looking" functions suffice.
- We will discuss weaker, efficiently implementable hash functions (in particular, universal hash functions) next week.
- Our analysis will work with these weaker hash functions.

But, we will make our lives easier by **assuming** we have access to a uniformly random hash function. This is an assumption we will use in future lectures as well. The assumption is often made in reseach papers even.

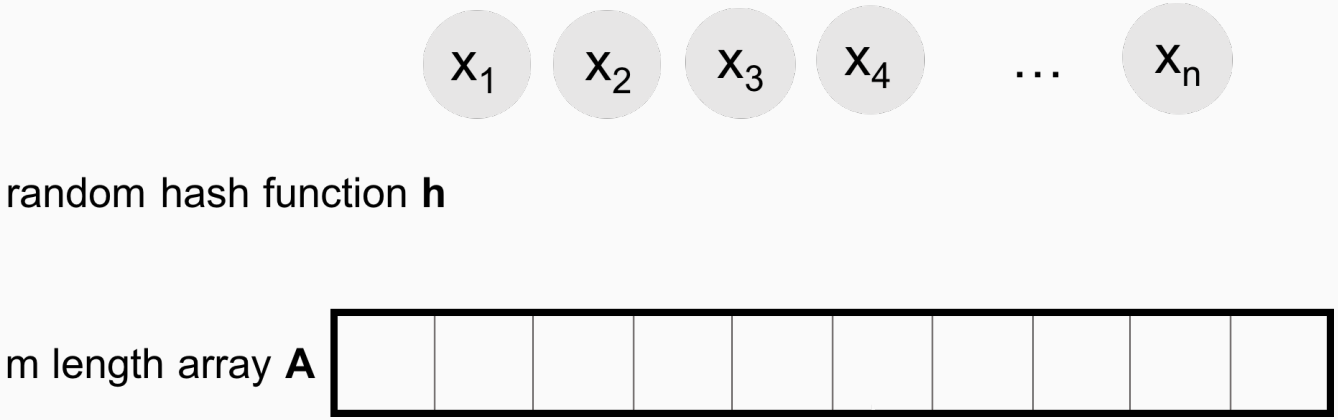
Count-Min Sketch



random hash function **h**

m length array **A**

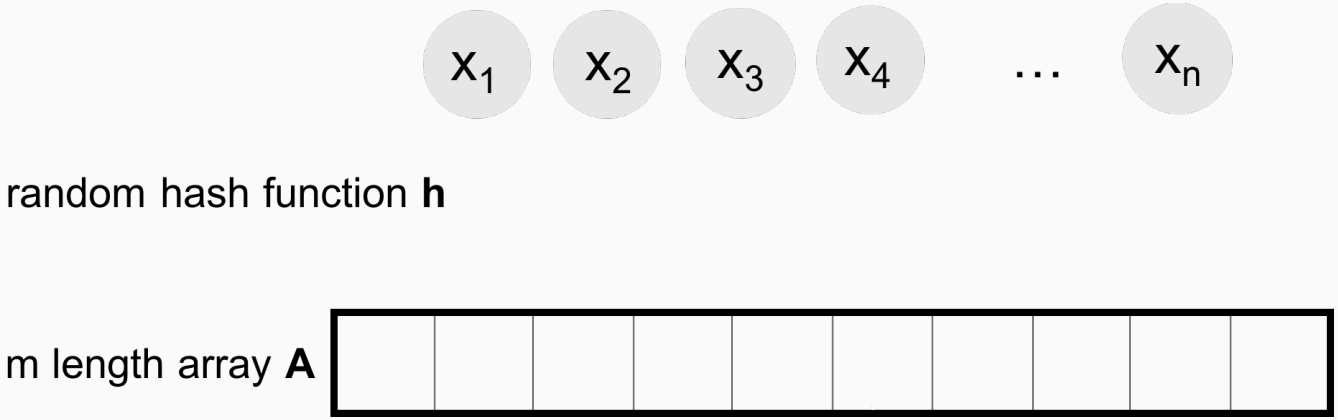
Count-Min Sketch



Count-Min Update:

- Choose random hash function h mapping to $\{1, \ldots, m\}$.
- For $i = 1, \ldots, n$
  - Given item x_i, set $\mathbf{A}[h(x_i)] = \mathbf{A}[h(x_i)] + 1$

Count-Min Sketch



Count-Min Update:

- Choose random hash function h mapping to $\{1, \ldots, m\}$.
- For $i = 1, \ldots, n$
  - Given item x_i, set $\mathbf{A}[h(x_i)] = \mathbf{A}[h(x_i)] + 1$

This algorithm can overcount the frequency of items, but it will never undercount.

Count-Min Sketch

We want to estimate the frequency of item v,

$$f(v) = \sum_{i=1}^{n} \mathbb{1}[x_i = v]$$

Count-Min Sketch

We want to estimate the frequency of item v,

$$f(v) = \sum_{i=1}^{n} \mathbb{1}[x_i = v]$$

To do this using our small space "sketch" **A**, return

$$\tilde{f}(v) = A[\mathbf{h}(v)]$$

Count-Min Sketch

We want to estimate the frequency of item v,

$$f(v) = \sum_{i=1}^{n} \mathbb{1}[x_i = v]$$

To do this using our small space "sketch" $\mathbf{A}$, return

$$\tilde{f}(v) = A[\mathbf{h}(v)]$$

Claim 1: We always have $\tilde{f}(v) = \mathbf{A}[\mathbf{h}(v)] \geq f(v)$. Why?

Count-Min Sketch

We want to estimate the frequency of item v,

$$f(v) = \sum_{i=1}^{n} \mathbb{1}[x_i = v]$$

To do this using our small space "sketch" $\mathbf{A}$, return

$$\tilde{f}(v) = A[\mathbf{h}(v)]$$

Claim 1: We always have $\tilde{f}(v) = \mathbf{A}[\mathbf{h}(v)] \geq f(v)$. Why?

$$\tilde{f}(v) = \mathbf{A}[\mathbf{h}(v)] = \sum_{i=1}^{n} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]$$

Count-Min Sketch

We want to estimate the frequency of item v,

$$f(v) = \sum_{i=1}^{n} \mathbb{1}[x_i = v]$$

To do this using our small space "sketch" $\mathbf{A}$, return

$$\tilde{f}(v) = A[\mathbf{h}(v)]$$

Claim 1: We always have $\tilde{f}(v) = \mathbf{A}[\mathbf{h}(v)] \geq f(v)$. Why?

$$\tilde{f}(v) = \mathbf{A}[\mathbf{h}(v)] = \sum_{i=1}^{n} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]$$

$$= f(v) + \sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]$$

Count-Min Sketch Accuracy

$$\mathbf{A}[\mathbf{h}(v)] = f(v) + \underbrace{\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]}_{\text{error in frequency estimate}}$$

Expected Error:

Count-Min Sketch Accuracy

$$\mathbf{A}[\mathbf{h}(v)] = f(v) + \underbrace{\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]}_{\text{error in frequency estimate}}$$

Expected Error:

$$\mathbb{E}\left[\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]\right] = \sum_{i:x_i \neq v} \Pr[\mathbf{h}(x_i) = \mathbf{h}(v)]$$

Count-Min Sketch Accuracy

$$\mathbf{A}[\mathbf{h}(v)] = f(v) + \underbrace{\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]}_{\text{error in frequency estimate}}$$

Expected Error:

$$\mathbb{E}\left[\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]\right] = \sum_{i:x_i \neq v} \Pr[\mathbf{h}(x_i) = \mathbf{h}(v)]$$

$$= \frac{n - f(v)}{m}$$

Count-Min Sketch Accuracy

$$\mathbf{A}[\mathbf{h}(v)] = f(v) + \underbrace{\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]}_{\text{error in frequency estimate}}$$

Expected Error:

$$\begin{aligned}
\mathbb{E}\left[\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]\right] &= \sum_{i:x_i \neq v} \Pr[\mathbf{h}(x_i) = \mathbf{h}(v)] \\
&= \frac{n - f(v)}{m} \\
&\leq \frac{n}{m}
\end{aligned}$$

Count-Min Sketch Accuracy

$$\mathbf{A}[\mathbf{h}(v)] = f(v) + \underbrace{\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]}_{\text{error in frequency estimate}}$$

Expected Error:

$$\mathbb{E}\left[\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]\right] = \frac{n - f(v)}{m} \leq \frac{n}{m}$$

What is a bound on probability that the error is $\geq \frac{2n}{m}$?

Count-Min Sketch Accuracy

$$\mathbf{A}[\mathbf{h}(v)] = f(v) + \underbrace{\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]}_{\text{error in frequency estimate}}$$

Expected Error:

$$\mathbb{E}\left[\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)]\right] = \frac{n - f(v)}{m} \leq \frac{n}{m}$$

What is a bound on probability that the error is $\geq \frac{2n}{m}$?

Markov's inequality: $\Pr\left[\sum_{i:x_i \neq v} \mathbb{1}[\mathbf{h}(x_i) = \mathbf{h}(v)] \geq \frac{2n}{m}\right] \leq 1/2$

$f(v)$: frequency of v in the stream. h: random hash function. m: size of Count-Min sketch array.

Count-Min Sketch Accuracy

Claim: For any v, with probability at least $1/2$,

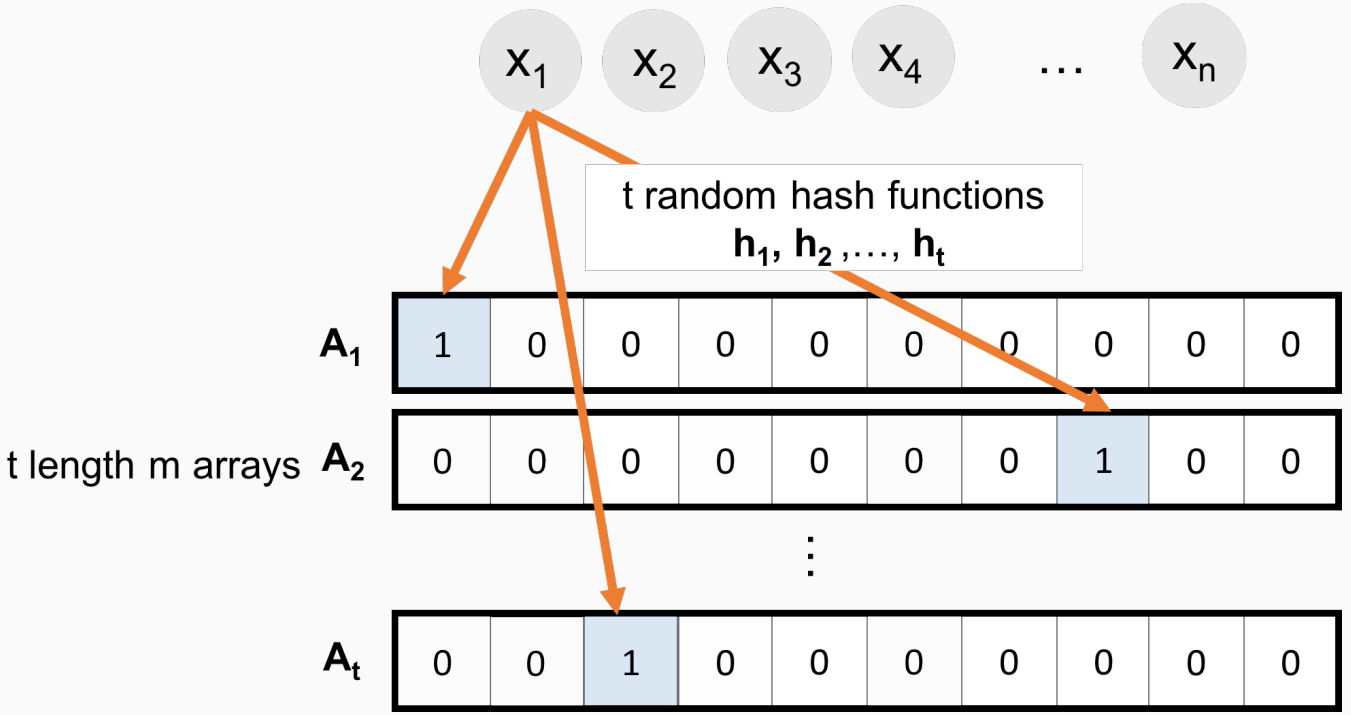
$$f(v) \leq \mathbf{A}[\mathbf{h}(v)] \leq f(v) + \frac{2n}{m}.$$

To solve the point query problem with error $\frac{\epsilon}{k}n$, set $m = k/\epsilon$

How can we improve the success probability?

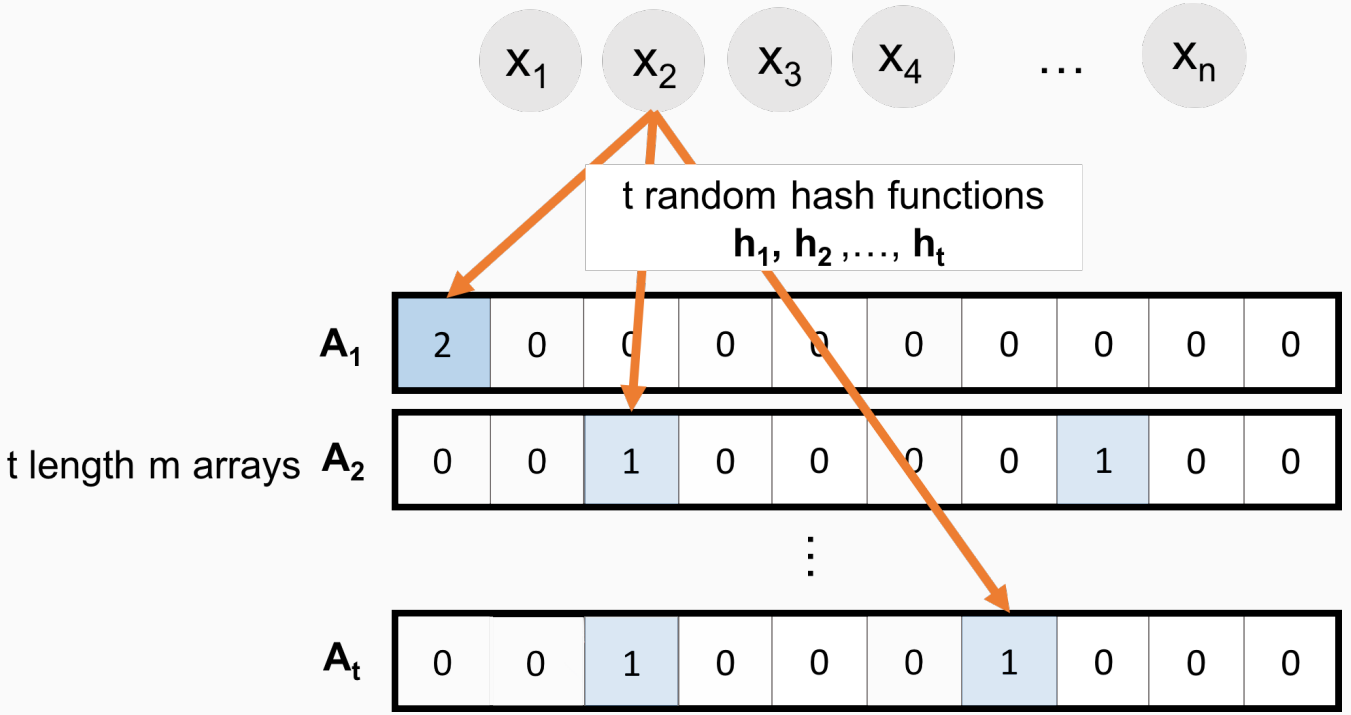
$f(v)$: frequency of v in the stream. h: random hash function. m: size of Count-Min sketch array.

Count-Min Sketch Accuracy



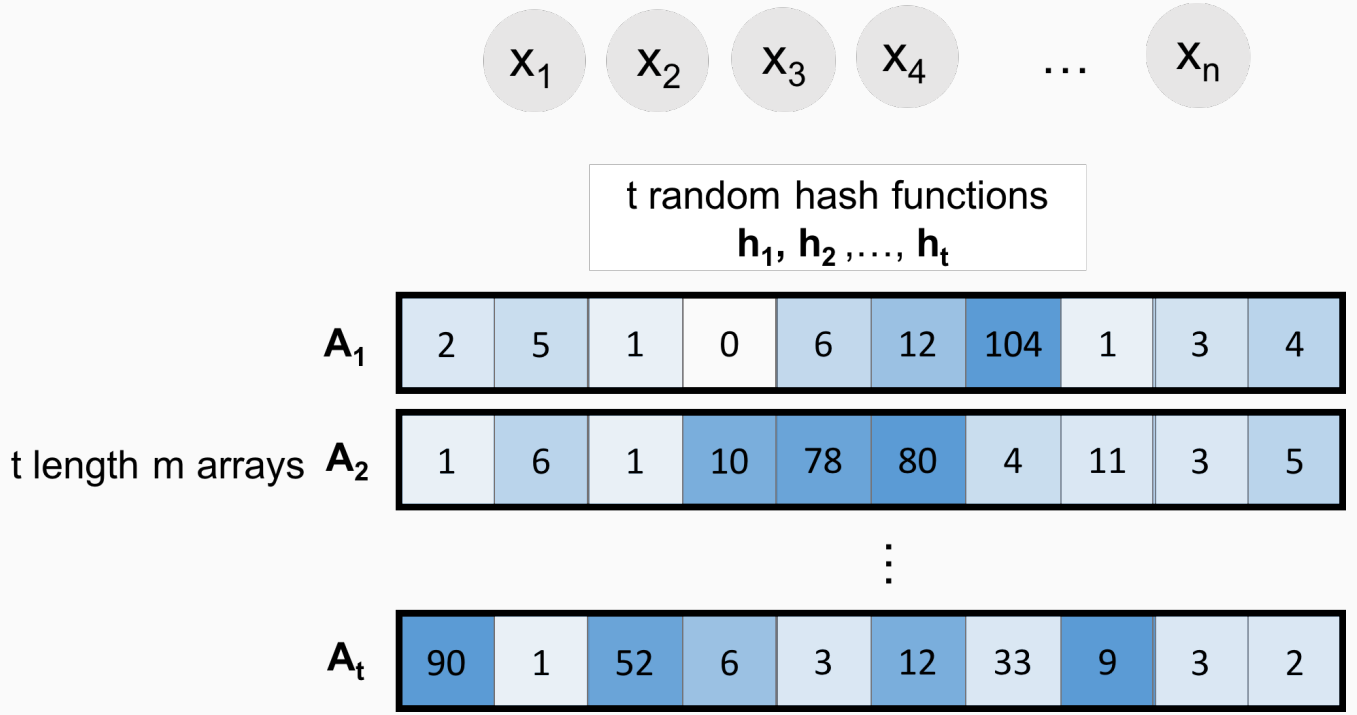
$f(v)$: frequency of v in the stream. $h_1, \ldots, h_t$: multiple random hash functions. m: size of t Count-Min sketch arrays.

Count-Min Sketch Accuracy



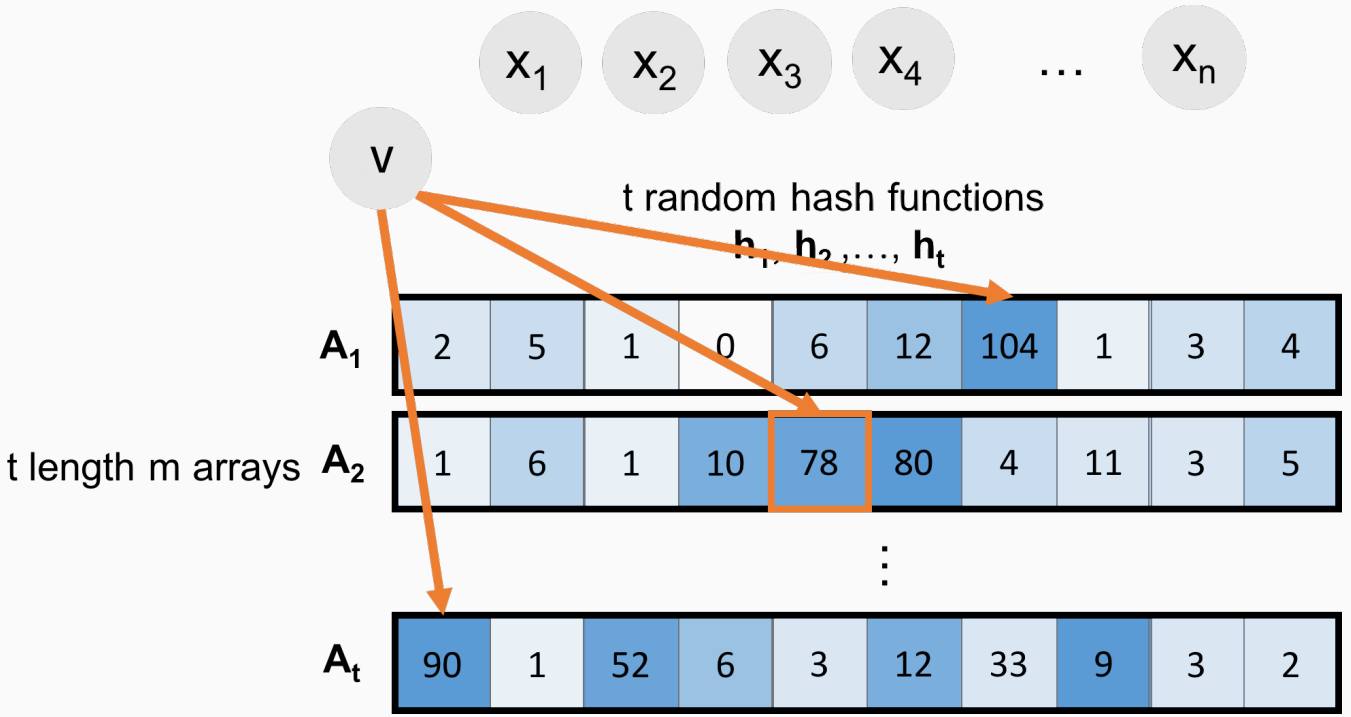
$f(v)$: frequency of v in the stream. $h_1, \ldots, h_t$: multiple random hash functions. m: size of t Count-Min sketch arrays.

Count-Min Sketch Accuracy



$f(v)$: frequency of v in the stream. $h_1, \ldots, h_t$: multiple random hash functions. m: size of t Count-Min sketch arrays.

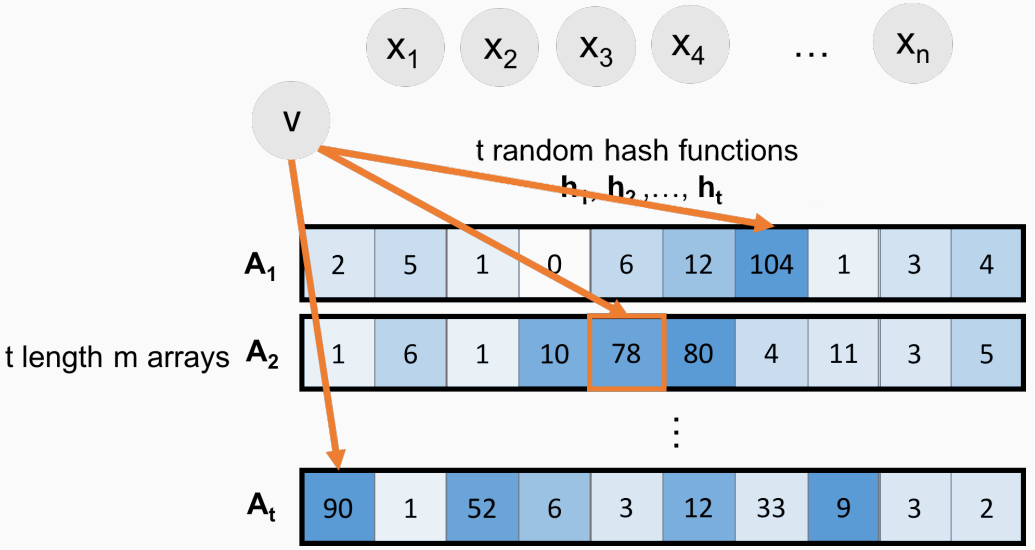
Count-Min Sketch Accuracy



Estimate $f(v)$ with $\tilde{f}(v) = \min_{i \in [t]} \mathbf{A}_i[\mathbf{h}_i(v)]$. (Count-**Min** sketch)

Why min instead of mean or median?

Count-Min Sketch Accuracy



Estimate $f(v)$ with $\tilde{f}(v) = \min_{i \in [t]} \mathbf{A}_i[\mathbf{h}_i(v)]$.

- For every v and i and $m = \frac{2k}{\epsilon}$, we know that with prob. $\geq 1/2$:
$$f(v) \leq \mathbf{A}_i[\mathbf{h}_i(v)] \leq f(v) + \frac{\epsilon n}{k}.$$
- $\Pr[f(v) \leq \tilde{f}(v) \leq f(v) + \frac{\epsilon n}{k}] \geq 1 - \frac{1}{2^t}$
- To get a good estimate with probability $\geq 1 - \delta$,
$$\textbf{set } t = \log(1/\delta)$$

Count-Min Sketch

Upshot: Count-Min sketch lets us estimate the frequency of each item in a stream up to error $\frac{\epsilon}{k}n$ with probability $\geq 1 - \delta$ in $O\left(\log(1/\delta) \cdot \frac{k}{\epsilon}\right)$ space.

Caveat: This is a for each v guarantee. We actually want a for all v guarantee: i.e. the bound should hold simultaneously for all v.

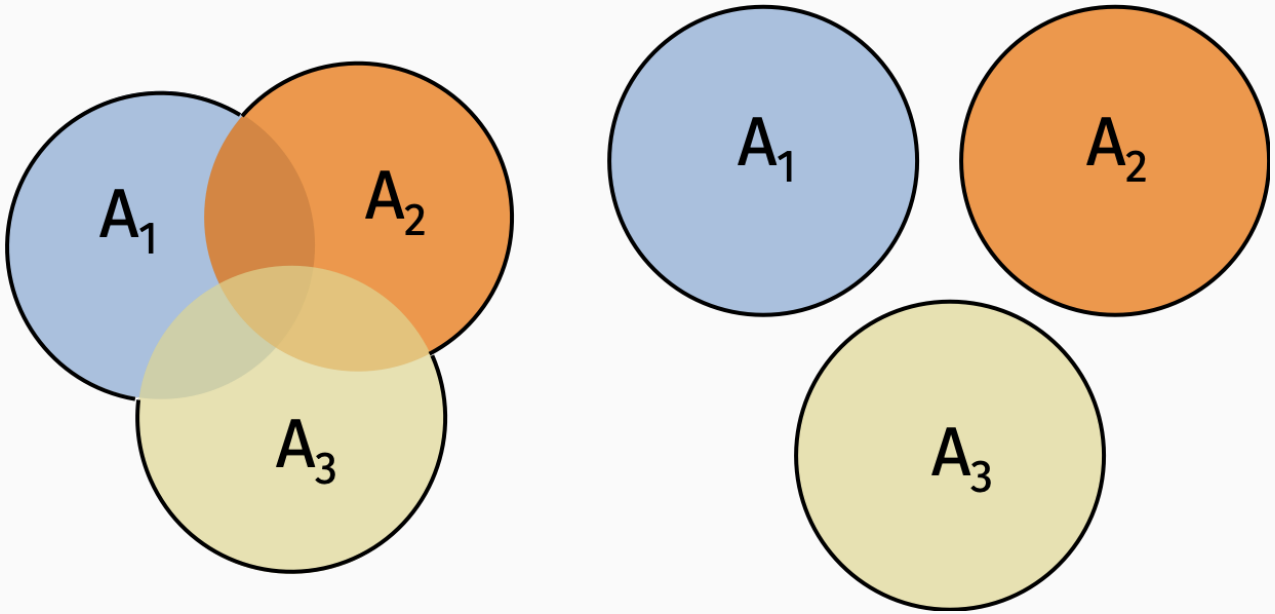
Use a Union Bound

Lemma (Union Bound)

For any random events $A_1, \ldots, A_k$:

$$\Pr[A_1 \cup A_2 \cup \ldots \cup A_k] \leq \Pr[A_1] + \Pr[A_2] + \ldots + \Pr[A_k].$$

Here $Pr[A_1 \cup A_2 \cup \ldots \cup A_k]$ means $Pr[A_1$ "or" $A_2 \ldots$ "or" $A_k]$



Proof by picture.

Use a Union Bound

The algorithm fails if $|f(v) - \tilde{f}(v)| > \frac{\epsilon}{k} n$ for any $v \in \{v_1, \ldots, v_u\}$.
By union bound:

$$\Pr[(\text{fail for } v_1) \text{ or } (\text{fail for } v_2) \text{ or} \ldots \text{ or} (\text{fail for } v_u)]$$

Use a Union Bound

The algorithm fails if $|f(v) - \tilde{f}(v)| > \frac{\epsilon}{k} n$ for any $v \in \{v_1, \ldots, v_u\}$.
By union bound:

$$\begin{aligned}
&\Pr[(\text{fail for } v_1) \text{ or } (\text{fail for } v_2) \text{ or} \ldots \text{ or} (\text{fail for } v_u)] \\
&\leq \Pr[(\text{fail for } v_1)] + \Pr[(\text{fail for } v_2)] \ldots + \Pr[(\text{fail for } v_u)]
\end{aligned}$$

Use a Union Bound

The algorithm fails if $|f(v) - \tilde{f}(v)| > \frac{\epsilon}{k} n$ for any $v \in \{v_1, \ldots, v_u\}$.
By union bound:

$$\begin{aligned}
&\Pr[(\text{fail for } v_1) \text{ or } (\text{fail for } v_2) \text{ or} \ldots \text{ or} (\text{fail for } v_u)] \\
&\leq \Pr[(\text{fail for } v_1)] + \Pr[(\text{fail for } v_2)] \ldots + \Pr[(\text{fail for } v_u)] \\
&\leq u \cdot \delta
\end{aligned}$$

Final Result

Set $\delta = \frac{1}{10u}$. With probability $9/10$, Count-Min sketch lets us estimate the frequency of all items in a stream up to error $\frac{\epsilon}{k}n$.

- Uses $O\left(\frac{k}{\epsilon}\log u\right)$ space.
- Accurate enough to solve the (ϵ, k)-Frequent elements problem – just return all v with estimated frequency $\geq n/k$.

Identifying Frequent Items

How do we identify the frequent items without having to look up the estimated frequency for all elements in the stream?

One approach:

- When a new item comes in at step i, check if its estimated frequency is $\geq i/k$ and store it if so.
- At step i remove any stored items whose estimated frequency drops below i/k.
- Store at most $O(k)$ items at once and have all items with estimated frequency $\geq n/k$ stored at the end of the stream.